0208. Implement Trie (Prefix Tree)

edium | Category: Tries | LeetCode: https://leetcode.com/problems/implement-trie

Problem

- Implement insert(word), search(word) (exact match) and startsWith(prefix) for lowercase words.

Intuition

- A trie stores strings character by character along a path from the root; shared prefixes share nodes.
- Each node has up to 26 children and an end-of-word flag.

Brute Force

- A HashSet handles search in O(L) but startsWith needs a scan of every word: O(N L).

Optimal Approach

- Insert: walk/create children for each char; mark the last node as end.
- Find(s): walk children; return null if missing.
- Search = find(word) != null && end; startsWith = find(prefix) != null.

Worked Example

```
insert "apple"  root-a-p-p-l-e*
search "apple"  -> true
search "app"    -> node exists but not end -> false
startsWith "app"-> true
insert "app"    marks the second p as end -> search "app" true
```

Java Solution

```java
public class Trie {
    private static class Node {
        Node[] next = new Node[26];
        boolean end;
    }

    private final Node root = new Node();

    public void insert(String word) {
        Node n = root;
        for (char c : word.toCharArray()) {
            if (n.next[c - 'a'] == null) n.next[c - 'a'] = new Node();
            n = n.next[c - 'a'];
        }
        n.end = true;
    }

    public boolean search(String word) {
        Node n = find(word);
        return n != null && n.end;
    }

    public boolean startsWith(String prefix) { return find(prefix) != null; }

    private Node find(String s) {
        Node n = root;
        for (char c : s.toCharArray()) {
            n = n.next[c - 'a'];
            if (n == null) return null;
        }
        return n;
    }
}
```

Complexity

- Each operation O(L). Space O(total characters 26) worst case.

Pitfalls

- Search must check the end flag; startsWith must not.